

Program No: 9

File Name:

Ex. No:

Date: _____

Create a to-do list application using React.js with Redux for state management.

AIM:

To create a simple, interactive to-do list web application using React and Redux for managing tasks.

ALGORITHM:

Step 1: Load the HTML file in a modern web browser to initialize the React app with Redux store.

Step 2: Enter a new to-do item in the input field and click "Add" to append it to the list.

Step 3: Check the checkbox next to a to-do item to mark it as completed, applying strikethrough styling.

Step 4: Double-click on a to-do item's text or click "Edit" to enter edit mode and modify the text.

Step 5: In edit mode, update the draft text and press Enter or click "Save" to confirm changes.

Step 6: Click "Cancel" in edit mode to discard changes and return to view mode.

Step 7: Click "Delete" next to a to-do item to remove it permanently from the list.

Step 8: Click "Clear Completed" to remove all completed to-do items at once.

Step 9: The app persists state in memory via Redux, updating the UI reactively on each action

PROGRAM:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <title>To-Do List</title>

  <script src="https://unpkg.com/react@18/umd/react.development.js"></script>

  <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>

  <script src="https://unpkg.com/redux@4.2.1/dist/redux.js"></script>

  <script src="https://unpkg.com/react-redux@8.0.5/dist/react-redux.js"></script>

  <script src="https://unpkg.com/@reduxjs/toolkit@1.9.5/dist/redux-toolkit.umd.js"></script>

  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>

  <style>

    body { font-family: Arial; background: #f7f7f8; padding: 20px; }

    .app { max-width: 600px; margin: auto; background: white; padding: 20px; border-radius:
8px; box-shadow: 0 0 10px rgba(0,0,0,0.1); }

    ul { list-style: none; padding: 0; }

    li { display: flex; align-items: center; padding: 8px; border-bottom: 1px solid #eee; }

    .completed { text-decoration: line-through; opacity: 0.7; }

    button { margin: 0 4px; padding: 4px 8px; border: none; border-radius: 4px; cursor: pointer;
}

    .add { background: #1976d2; color: white; }

    .clear { background: #9e9e9e; }

    .edit { background: #ffd54f; }

    .delete { background: #ef5350; color: white; }

    .save { background: #4caf50; color: white; }

    .cancel { background: #9e9e9e; }

  </style>

</head>
```

```
<body>

<div id="root"></div>

<script type="text/babel">

  const { useState } = React;

  const { Provider, useSelector, useDispatch } = ReactRedux;

  const { configureStore, createSlice, nanoid } = RTK;


  const todosSlice = createSlice({

    name: "todos",

    initialState: [],

    reducers: {

      addTodo: (state, action) => { state.unshift({ id: nanoid(), text: action.payload.trim(),
completed: false }); },

      toggleTodo: (state, action) => { const t = state.find(t => t.id === action.payload); if (t)
t.completed = !t.completed; },

      deleteTodo: (state, action) => state.filter(t => t.id !== action.payload),

      editTodo: (state, action) => { const t = state.find(t => t.id === action.payload.id); if (t)
t.text = action.payload.text.trim(); },

      clearCompleted: (state) => state.filter(t => !t.completed),

    },

  });

  const { addTodo, toggleTodo, deleteTodo, editTodo, clearCompleted } = todosSlice.actions;

  const store = configureStore({ reducer: todosSlice.reducer });

  function App() {

    const [text, setText] = useState("");

    const [editing, setEditing] = useState(null);

    const [draft, setDraft] = useState("");

    const todos = useSelector(state => state);

    const dispatch = useDispatch();

    const add = (e) => { e.preventDefault(); if (text.trim()) { dispatch(addTodo(text));
```

```
setText(""); } };
```

```
const save = () => { if (draft.trim()) dispatch(editTodo({ id: editing, text: draft })); setEditing(null); };
```

```
return (
```

```
  <div className="app">
```

```
    <h1>To-Do List</h1>
```

```
    <form onSubmit={add}>
```

```
      <input value={text} onChange={e => setText(e.target.value)} placeholder="Add to-  
do..." />
```

```
      <button className="add" type="submit">Add</button>
```

```
      <button      className="clear"      type="button"      onClick={()      =>  
dispatch(clearCompleted())}>Clear Completed</button>
```

```
    </form>
```

```
    <ul>
```

```
      {todos.map(t => (
```

```
        <li key={t.id} className={t.completed ? "completed" : ""}>
```

```
          <input      type="checkbox"      checked={t.completed}      onChange={()      =>  
dispatch(toggleTodo(t.id))} />
```

```
          {editing === t.id ? (
```

```
            ◇
```

```
            <input value={draft} onChange={e => setDraft(e.target.value)} onKeyDown={e  
=> e.key === "Enter" && save()} autoFocus />
```

```
            <button className="save" onClick={save}>Save</button>
```

```
            <button className="cancel" onClick={() => setEditing(null)}>Cancel</button>
```

```
          </>
```

```
        ) : (
```

```
          ◇
```

```
          <span      onDoubleClick={()      =>      {      setEditing(t.id);      setDraft(t.text);  
}}>{t.text}</span>
```

```
          <button className="edit" onClick={() => { setEditing(t.id); setDraft(t.text);  
}}>Edit</button>
```

```
        <button          className="delete"          onClick={()          =>
dispatch(deleteTodo(t.id))}>Delete</button>

      </>

    })

  </li>

  )})

</ul>

</div>

);

}

ReactDOM.createRoot(document.getElementById("root")).render(<Provider
store={store}><App /></Provider>);

</script>

</body>

</html>
```

OUTPUT:

The image displays two sequential screenshots of a web-based To-Do List application. Both screenshots feature a light gray header with the title 'To-Do List' in a large, bold, black font. Below the header, there is a white input field with the placeholder text 'Add to-do...', a blue 'Add' button, and a gray 'Clear Completed' button.

The top screenshot shows the application in its initial state, with no tasks listed. The bottom screenshot shows the application after two tasks have been added. The tasks are listed below the input field, separated by a horizontal line. Each task is preceded by an unchecked checkbox. The first task is 'task 2' and the second is 'task 1'. To the right of each task name are two buttons: a yellow 'Edit' button and a red 'Delete' button. The tasks are ordered from top to bottom as 'task 2' and then 'task 1'.

RESULT:

Thus the program was successfully executed.